

Item 2: Consider a builder when faced with many constructor parameters

Static factories and constructors share a limitation: they do not scale well to large numbers of optional parameters. Consider the case of a class representing the Nutrition Facts label that appears on packaged foods. These labels have a few required fields—serving size, servings per container, and calories per serving—and over twenty optional fields—total fat, saturated fat, trans fat, cholesterol, sodium, and so on. Most products have nonzero values for only a few of these optional fields.

What sort of constructors or static factories should you write for such a class? Traditionally, programmers have used the *telescoping constructor* pattern, in which you provide a constructor with only the required parameters, another with a single optional parameter, a third with two optional parameters, and so on, culminating in a constructor with all the optional parameters. Here’s how it looks in practice. For brevity’s sake, only four optional fields are shown:

```
// Telescoping constructor pattern - does not scale well!
public class NutritionFacts {
    private final int servingSize; // (mL)           required
    private final int servings;    // (per container) required
    private final int calories;    //                optional
    private final int fat;         // (g)             optional
    private final int sodium;     // (mg)            optional
    private final int carbohydrate; // (g)          optional

    public NutritionFacts(int servingSize, int servings) {
        this(servingSize, servings, 0);
    }

    public NutritionFacts(int servingSize, int servings,
        int calories) {
        this(servingSize, servings, calories, 0);
    }

    public NutritionFacts(int servingSize, int servings,
        int calories, int fat) {
        this(servingSize, servings, calories, fat, 0);
    }

    public NutritionFacts(int servingSize, int servings,
        int calories, int fat, int sodium) {
        this(servingSize, servings, calories, fat, sodium, 0);
    }

    public NutritionFacts(int servingSize, int servings,
        int calories, int fat, int sodium, int carbohydrate) {
        this.servingSize = servingSize;
        this.servings    = servings;
        this.calories     = calories;
        this.fat          = fat;
        this.sodium       = sodium;
        this.carbohydrate = carbohydrate;
    }
}
```

When you want to create an instance, you use the constructor with the shortest parameter list containing all the parameters you want to set:

```
NutritionFacts cocaCola =
    new NutritionFacts(240, 8, 100, 0, 35, 27);
```

Typically this constructor invocation will require many parameters that you don’t want to set, but you’re forced to pass a value for them anyway. In this case, we passed a value of 0 for fat. With “only” six parameters this may not seem so bad, but it quickly gets out of hand as the number of parameters increases.

In short, **the telescoping constructor pattern works, but it is hard to write client code when there are many parameters, and harder still to read it.** The reader is left wondering what all those values mean and must carefully count parameters to find out. Long sequences of identically typed parameters can cause subtle bugs. If the client accidentally reverses two such parameters, the compiler won’t complain, but the program will misbehave at runtime (Item 40).

A second alternative when you are faced with many constructor parameters is the *JavaBeans* pattern, in which you call a parameterless constructor to create the object and then call setter methods to set each required parameter and each optional parameter of interest:

```
// JavaBeans Pattern - allows inconsistency, mandates mutability
public class NutritionFacts {
```

```

// Parameters initialized to default values (if any)
private int servingSize = -1; // Required; no default value
private int servings    = -1; //      "      "      "      "
private int calories    = 0;
private int fat          = 0;
private int sodium      = 0;
private int carbohydrate = 0;

public NutritionFacts() { }
// Setters
public void setServingSize(int val) { servingSize = val; }
public void setServings(int val)   { servings    = val; }
public void setCalories(int val)   { calories    = val; }
public void setFat(int val)        { fat         = val; }
public void setSodium(int val)     { sodium      = val; }
public void setCarbohydrate(int val) { carbohydrate = val; }
}

```

This pattern has none of the disadvantages of the telescoping constructor pattern. It is easy, if a bit wordy, to create instances, and easy to read the resulting code:

```

NutritionFacts cocaCola = new NutritionFacts();
cocaCola.setServingSize(240);
cocaCola.setServings(8);
cocaCola.setCalories(100);
cocaCola.setSodium(35);
cocaCola.setCarbohydrate(27);

```

Unfortunately, the JavaBeans pattern has serious disadvantages of its own. Because construction is split across multiple calls, **a JavaBean may be in an inconsistent state partway through its construction**. The class does not have the option of enforcing consistency merely by checking the validity of the constructor parameters. Attempting to use an object when it's in an inconsistent state may cause failures that are far removed from the code containing the bug, hence difficult to debug. A related disadvantage is that **the JavaBeans pattern precludes the possibility of making a class immutable** (Item 15), and requires added effort on the part of the programmer to ensure thread safety.

It is possible to reduce these disadvantages by manually “freezing” the object when its construction is complete and not allowing it to be used until frozen, but this variant is unwieldy and rarely used in practice. Moreover, it can cause errors at runtime, as the compiler cannot ensure that the programmer calls the freeze method on an object before using it.

Luckily, there is a third alternative that combines the safety of the telescoping constructor pattern with the readability of the JavaBeans pattern. It is a form of the *Builder* pattern [Gamma95, p. 97]. Instead of making the desired object directly, the client calls a constructor (or static factory) with all of the required parameters and gets a *builder object*. Then the client calls setter-like methods on the builder object to set each optional parameter of interest. Finally, the client calls a parameterless `build` method to generate the object, which is immutable. The builder is a static member class (Item 22) of the class it builds. Here's how it looks in practice:

```

// Builder Pattern
public class NutritionFacts {
    private final int servingSize;
    private final int servings;
    private final int calories;
    private final int fat;
    private final int sodium;
    private final int carbohydrate;

    public static class Builder {
        // Required parameters
        private final int servingSize;
        private final int servings;

        // Optional parameters - initialized to default values
        private int calories = 0;
        private int fat      = 0;
        private int carbohydrate = 0;
        private int sodium    = 0;

        public Builder(int servingSize, int servings) {
            this.servingSize = servingSize;
            this.servings    = servings;
        }

        public Builder calories(int val)
            { calories = val;      return this; }
        public Builder fat(int val)
            { fat = val;          return this; }
        public Builder carbohydrate(int val)

```

```

        { carbohydrate = val; return this; }
    public Builder sodium(int val)
    { sodium = val; return this; }

    public NutritionFacts build() {
        return new NutritionFacts(this);
    }
}

private NutritionFacts(Builder builder) {
    servingSize = builder.servingSize;
    servings    = builder.servings;
    calories    = builder.calories;
    fat         = builder.fat;
    sodium      = builder.sodium;
    carbohydrate = builder.carbohydrate;
}
}

```

Note that `NutritionFacts` is immutable, and that all parameter default values are in a single location. The builder's setter methods return the builder itself so that invocations can be chained. Here's how the client code looks:

```

NutritionFacts cocaCola = new NutritionFacts.Builder(240, 8).
    calories(100).sodium(35).carbohydrate(27).build();

```

This client code is easy to write and, more importantly, to read. **The Builder pattern simulates named optional parameters** as found in Ada and Python.

Like a constructor, a builder can impose invariants on its parameters. The `build` method can check these invariants. It is critical that they be checked after copying the parameters from the builder to the object, and that they be checked on the object fields rather than the builder fields (Item 39). If any invariants are violated, the `build` method should throw an `IllegalStateException` (Item 60). The exception's detail method should indicate which invariant is violated (Item 63).

Another way to impose invariants involving multiple parameters is to have setter methods take entire groups of parameters on which some invariant must hold. If the invariant isn't satisfied, the setter method throws an `IllegalArgumentException`. This has the advantage of detecting the invariant failure as soon as the invalid parameters are passed, instead of waiting for `build` to be invoked.

A minor advantage of builders over constructors is that builders can have multiple varargs parameters. Constructors, like methods, can have only one varargs parameter. Because builders use separate methods to set each parameter, they can have as many varargs parameters as you like, up to one per setter method.

The Builder pattern is flexible. A single builder can be used to build multiple objects. The parameters of the builder can be tweaked between object creations to vary the objects. The builder can fill in some fields automatically, such as a serial number that automatically increases each time an object is created.

A builder whose parameters have been set makes a fine *Abstract Factory* [Gamma95, p. 87]. In other words, a client can pass such a builder to a method to enable the method to create one or more objects for the client. To enable this usage, you need a type to represent the builder. If you are using release 1.5 or a later release, a single generic type (Item 26) suffices for *all* builders, no matter what type of object they're building:

```

// A builder for objects of type T
public interface Builder<T> {
    public T build();
}

```

Note that our `NutritionFacts.Builder` class could be declared to implement `Builder<NutritionFacts>`.

Methods that take a `Builder` instance would typically constrain the builder's type parameter using a *bounded wildcard type* (Item 28). For example, here is a method that builds a tree using a client-provided `Builder` instance to build each node:

```

Tree buildTree(Builder<? extends Node> nodeBuilder) { ... }

```

The traditional `Abstract Factory` implementation in Java has been the `Class` object, with the `newInstance` method playing the part of the `build` method. This usage is fraught with problems. The `newInstance` method always attempts to invoke the class's parameterless constructor, which may not even exist. You don't get a compile-time error if the class has no accessible parameterless constructor. Instead, the client code must cope with `InstantiationException` or `IllegalAccessException` at runtime, which is ugly and inconvenient. Also, the `newInstance` method propagates any exceptions thrown by the parameterless constructor, even though `newInstance` lacks the corresponding `throws` clauses. In other words, **`Class.newInstance` breaks compile-time exception checking**. The `Builder` interface, shown above, corrects these deficiencies.

The Builder pattern does have disadvantages of its own. In order to create an object, you must first create its builder. While the cost of creating the builder is unlikely to be noticeable in practice, it could be a problem in some performance-critical situations. Also, the Builder pattern is more verbose than the telescoping constructor pattern, so it should be used only if there are enough parameters, say, four or more. But keep in mind that you may want to add parameters in the future. If you start out with constructors or static factories, and add a builder when the class evolves to the point where the number of parameters starts to get out of hand, the obsolete constructors or static factories will stick out like a sore thumb. Therefore, it's often better to start with a builder in the first place.

In summary, **the Builder pattern is a good choice when designing classes whose constructors or static factories would have more than a handful of parameters**, especially if most of those parameters are optional. Client code is much easier to read and write with builders than with the traditional telescoping constructor pattern, and builders are much safer than JavaBeans.